

Capítulo 1

Matrizes

Em geral, podemos ter uma matriz n -dimensional $M[i_1, i_2, \dots, i_n]$, com índices i_1 variando de c_1 a f_1 , i_2 de c_2 a f_2 , $\dots$, e i_n variando de c_n a f_n . Interessa-nos a maneira de se ter acesso aos dados armazenados em uma matriz e não as possíveis operações matemáticas realizáveis com matrizes. As operações fundamentais que nos interessam são a atribuição de um valor a um elemento da matriz e a consulta de um elemento da matriz.

A operação `consul(M,j)` ($c \leq j \leq f$) nos fornece o valor $M[j]$ armazenado na posição j da matriz M . A operação `atrib(M,j,v)` atribui o valor v para o elemento $M[j]$. Na atribuição há duas possibilidades:

1. a matriz M é alterada localmente; ou
2. o resultado é uma nova matriz N , com todos os elementos iguais aos de M , exceto aquele da posição j .

A seguir são discutidas as implementações desses dois procedimentos para algumas classes de matrizes.

1.1 Matrizes unidimensionais

Dada uma matriz unidimensional M ($M[i]$, $i = c, \dots, f$), existem dois modos de se representar M : a representação natural e a “baseada”. Na natural, M é representada por um objeto `vetor[c, ..., f]` do tipo `t`, onde `t` é o tipo dos elementos da matriz M . Essa mesma matriz na representação “baseada” é representada pelo objeto `vetor[b, ..., b+(f-c)]` do tipo `t`.

A seguir estão listados os procedimentos escritos na linguagem C para a consulta e atribuição na representação natural, considerando-se o tipo `t` como inteiro (`int`):

```
1 int atrib_n( int* M,
2             int i,
3             int c,
4             int f,
5             int v)
6 {
7     if ((c <= i) && (i <= f))
8         M[i-c] = v;
9     else
10    {
11        printf( "Indice %d
12                invalido!!!", i );
13        exit( EXIT_FAILURE );
14    }
```

```
1 int consul_n( int* M,
2              int i,
3              int c,
4              int f)
5 {
6     if ((c <= i) && (i <= f))
7         return M[i-c];
8     else
9     {
10        printf( "Indice %d
11                invalido!!!", i );
12        exit( EXIT_FAILURE );
13    }
```

Na representação “baseada” os procedimentos são:

```

1 int atrib_b( int* M,
2             int i,
3             int b,
4             int c,
5             int f,
6             int v)
7 {
8     if ((c <= i) && (i <= f))
9         M[b+(i-c)] = v;
10    else
11    {
12        printf( "Indice %d
13                inválido!!!", i );
14        exit( EXIT_FAILURE );
15    }
16 }
```

```

1 int consul_b( int* M,
2              int i,
3              int b,
4              int c,
5              int f)
6 {
7     if ((c <= i) && (i <= f))
8         return M[b+(i-c)];
9     else
10    {
11        printf( "Indice %d
12                inválido!!!", i );
13        exit( EXIT_FAILURE );
14    }
15 }
```

Total de elementos (T) e índice k de um elemento na representação “baseada”, dada a posição $c \leq i \leq f$:

- $T = f - c + 1$;
- $k = b + i - c$.

1.2 Matrizes gerais

No caso geral de uma matriz n -dimensional $M[i_1, i_2, \dots, i_n]$, com índices i_1 variando de c_1 a f_1 , i_2 de c_2 a f_2 , ..., e i_n variando de c_n a f_n , na representação natural é usado um objeto **vetor** $[c_1, \dots, f_1]$ do tipo **vetor** $[c_2, \dots, f_2]$ do tipo ... **vetor** $[c_n, \dots, f_n]$ do tipo **t**. Os procedimentos de consulta e atribuição são semelhantes aos do caso de matrizes unidimensionais, com a ressalva de que deve-se verificar a validade de todos os índices.

1.2.1 Matriz bidimensional

Em uma matriz bidimensional $M[i_1 = c_1, \dots, f_1; i_2 = c_2, \dots, f_2]$, existem:

- $\ell = f_2 - c_2 + 1 \rightarrow$ linhas;
- $t_1 = f_1 - c_1 + 1 \rightarrow$ elementos em cada linha; e
- $t_2 = t_1 \cdot \ell = t_1 \cdot (f_2 - c_2 + 1) \rightarrow$ elementos na matriz.

O total de elementos em cada linha e na matriz pode ser representada como:

$$\begin{array}{ccccccc} (c_1, c_2) & \dots & (c_1, f_1) & \dots & (i_1, c_2) & \dots & (i_1, i_2) & \dots & (i_1, f_2) & \dots & (f_1, f_2) \\ \boxed{b} & & b + t_1 - 1 & & & & & & & & b + t_2 - 1 \end{array}$$

Logo, a representação “baseada” de M é feita por um objeto **vetor** $[b, \dots, b + t_2 - 1]$ do tipo **t** e o índice k correspondente à posição (i_1, i_2) é $k = b + t_1 \cdot (i_1 - c_1) + (i_2 - c_2)$.

1.2.2 Matriz tridimensional

Em uma matriz tridimensional $M[i_1 = c_1, \dots, f_1; i_2 = c_2, \dots, f_2; i_3 = c_3, \dots, f_3]$, existem:

- $p = f_1 - c_1 + 1 \rightarrow$ quantidade de planos;
- $t_1 = f_3 - c_3 + 1 \rightarrow$ elementos em cada linha de um plano;
- $t_2 = t_1 \cdot (f_2 - c_2 + 1) = (f_3 - c_3 + 1) \cdot (f_2 - c_2 + 1) \rightarrow$ elementos em cada plano; e
- $t_3 = t_2 \cdot (f_1 - c_1 + 1) = (f_3 - c_3 + 1) \cdot (f_2 - c_2 + 1) \cdot (f_1 - c_1 + 1) \rightarrow$ elementos na matriz.

Logo, a representação “baseada” de M é feita por um objeto `vetor` $[b, \dots, b + t_3 - 1]$ do tipo `t`, sendo que o plano C_1 corresponde às posições $b + t_1, \dots, (b + t_2) + t_2 - 1$ e assim sucessivamente.

Qual o índice k correspondente à posição (i_1, i_2, i_3) ? Antes do plano i_1 estão os planos $c_1, c_1 + 1, \dots, i_1 - 1$, ocupando $t_2 \cdot (i_1 - c_1)$ posições de memória. No plano i_1 , antes da linha i_2 , estão as linhas $c_2, c_2 + 1, \dots, i_2 - 1$, ocupando $t_1 \cdot (i_2 - c_2)$ posições de memória. Na linha i_2 , antes do elemento i_3 , estão os elementos $c_3, c_3 + 1, \dots, i_3 - 1$, ocupando $i_3 - c_3$ posições de memória. Logo:

$$\begin{aligned} k &= b + t_2 \cdot (i_1 - c_1) + t_1 \cdot (i_2 - c_2) + (i_3 - c_3) \\ &= b - (t_2 \cdot c_1 + t_1 \cdot c_2 + c_3) + (t_2 \cdot i_1 + t_1 \cdot i_2 + i_3). \end{aligned}$$

Note-se que t_1, t_2 e t_3 dependem apenas da estrutura da matriz e podem ser pré-calculados. Assim, se $\delta = b - (t_2 \cdot c_1 + t_1 \cdot c_2 + c_3)$, então $k = \delta + (t_2 \cdot i_1 + t_1 \cdot i_2 + i_3)$.

1.2.3 Matriz n -dimensional

Total de elementos (T) e índice de um elemento (ℓ) na representação “baseada”:

1D: $M[i_1]$ e $c_1 \leq i_1 \leq f_1$:

$$\begin{aligned} T &= f_1 - c_1 + 1; \\ k &= b + \\ &\quad (i_1 - c_1). \end{aligned}$$

2D: $M[i_1, i_2]$ e $c_1 \leq i_1 \leq f_1$ e $c_2 \leq i_2 \leq f_2$:

$$\begin{aligned} T &= (f_1 - c_1 + 1) \cdot (f_2 - c_2 + 1); \\ k &= b + \\ &\quad (f_1 - c_1 + 1) \cdot (i_2 - c_2) + \\ &\quad (i_1 - c_1). \end{aligned}$$

3D: $M[i_1, i_2, i_3]$ e $c_j \leq i_j \leq f_j, j = 1, 2, 3$:

$$\begin{aligned} T &= (f_1 - c_1 + 1) \cdot (f_2 - c_2 + 1) \cdot (f_3 - c_3 + 1); \\ k &= b + \\ &\quad (f_1 - c_1 + 1) \cdot (f_2 - c_2 + 1) \cdot (i_3 - c_3) + \\ &\quad (f_1 - c_1 + 1) \cdot (i_2 - c_2) + \\ &\quad (i_1 - c_1). \end{aligned}$$

4D: $M[i_1, i_2, i_3, i_4]$ e $c_j \leq i_j \leq f_j, j = 1, \dots, 4$:

$$\begin{aligned}
 T &= (f_1 - c_1 + 1) \cdot (f_2 - c_2 + 1) \cdot (f_3 - c_3 + 1) \cdot (f_4 - c_4 + 1); \\
 k &= b + \\
 &\quad (f_1 - c_1 + 1) \cdot (f_2 - c_2 + 1) \cdot (f_3 - c_3 + 1) \cdot (i_4 - c_4) + \\
 &\quad (f_1 - c_1 + 1) \cdot (f_2 - c_2 + 1) \cdot (i_3 - c_3) + \\
 &\quad (f_1 - c_1 + 1) \cdot (i_2 - c_2) + \\
 &\quad (i_1 - c_1).
 \end{aligned}$$

nD: $M[i_1, i_2, \dots, i_n]$ e $c_j \leq i_j \leq f_j, j = 1, \dots, n$:

$$\begin{aligned}
 T &= \prod_{j=1}^n (f_j - c_j + 1); \\
 k &= b + \sum_{r=n}^2 \left(\left(\prod_{j=1}^{r-1} (f_j - c_j + 1) \right) \cdot (i_r - c_r) \right) + (i_1 - c_1).
 \end{aligned}$$

1.3 Matrizes Especiais

1.3.1 Matrizes diagonais

Uma matriz bidimensional $M[i_1 = c_1, \dots, f_1; i_2 = c_2, \dots, f_2]$ é dita diagonal se todos os elementos com índices $i_1 \neq i_2$ são nulos. Ela é representada por um objeto `vetor[b, ..., b + p]` do tipo `t`, onde p é o mínimo entre $f_1 - c_1$ e $f_2 - c_2$. Um procedimento para consulta em uma matriz diagonal é:

```

1 int consul_d( int* M,
2               int  i,
3               int  j,
4               int  b,
5               int  c1,
6               int  f1,
7               int  c2,
8               int  f2)
9 {
10     if (((c1 <= i) && (i <= f1)) &&
11         ((c2 <= j) && (j <= f2)))
12         if (i == j){
13             if(f2-c2 < f1-c1)
14                 return M[b+(j-c2)];
15             return M[b+(i-c1)];
16         }
17         else
18             return 0;
19     else
20     {
21         printf( "Indice (%d,%d)
22                 invalido!!!", i, j );
23         exit( EXIT_FAILURE );
24     }
25 }

```

A atribuição de um valor não nulo a um elemento com $i \neq j$ resultaria numa matriz não diagonal, a qual necessitaria ser representada de outro modo.

1.3.2 Matrizes triangulares

Uma matriz bidimensional $M[i = 1, \dots, n; j = 1, \dots, n]$ é dita triangular inferior quando todos elementos com $i < j$ são nulos. Na primeira linha de M , todos os elementos são nulos, exceto talvez $M[1, 1]$. Na segunda linha, só $M[2, 1]$ e $M[2, 2]$ é que podem ser não nulos. Na linha i , podem ser não nulos os i elementos $M[i, 1], M[i, 2], \dots, M[i, i]$. Portanto, os $1 + 2 + \dots + n = n \cdot (n + 1)/2$ elementos que estão abaixo da diagonal principal da matriz é que podem ser não nulos.

Logo, M pode ser representada por um objeto `vetor[b, \dots, b + n \cdot (n + 1)/2]` do tipo `t`. Para se localizar um elemento $M[i, j]$ nesta representação note-se que antes dos elementos da linha i , há 1 elemento da primeira linha, 2 da segunda linha, $\dots$, $(i - 1)$ da $(i - 1)$ -ésima linha, totalizando $1 + 2 + \dots + (i - 1) = i \cdot (i - 1)/2$. Logo, o j -ésimo elemento da linha i ficará na posição $k = b + j + i \cdot (i - 1)/2$.

1.3.3 Matrizes simétricas e anti-simétricas

Uma matriz bidimensional $M[i = 1, \dots, n; j = 1, \dots, n]$ é simétrica quando $M[i, j] = M[j, i]$ para todo $i, j = 1, \dots, n$. Neste caso, é necessário armazenar apenas o triângulo inferior (superior) de M e para se obter $M[i, j]$, com $i < j$, deve-se acessar o elemento $M[j, i]$.

Uma matriz bidimensional $M[i = 1, \dots, n; j = 1, \dots, n]$ é anti-simétrica quando $M[i, j] = -M[j, i]$ para todo $i, j = 1, \dots, n$ e todos os elementos da diagonal principal são nulos. Neste caso, é necessário armazenar apenas o triângulo inferior de M (abaixo da diagonal principal), ou seja, 0 elementos da primeira linha, 1 da segunda linha, $\dots$, $(i - 1)$ da $(i - 1)$ -ésima linha, $\dots$, $(n - 1)$ da n -ésima linha. O total de elementos a serem armazenados é $0 + 1 + \dots + (n - 1) = n \cdot (n - 1)/2$.

1.3.4 Matrizes tridiagonais e faixas

Uma matriz bidimensional $M[i = 1, \dots, n; j = 1, \dots, n]$ é tridiagonal se todos os elementos são nulos, exceto os elementos $M[i, i]$ da diagonal principal e os elementos $M[i + 1, i]$ e $M[i, i + 1]$ das duas diagonais paralelas imediatamente vizinhas à diagonal principal. Neste caso, as linhas 1 e n conterão 2 elementos não nulos cada e as outras linhas conterão 3 elementos não nulos ($M[i, i]$, $M[i + 1, i]$ e $M[i, i + 1]$), totalizando $2 + 3 \cdot (n - 2) + 2 = 3 \cdot n - 2$ elementos.

1.4 Matrizes Esparsas

Uma matriz é dita esparsa quando a maior parte de seus elementos é nula. Pode-se economizar algumas células de memória ao se armazenar apenas os elementos não nulos. Pode-se representar uma matriz esparsa na linguagem C usando-se a estrutura definida a seguir:

```
1 typedef struct
2 {
3     int ind;
4     float val;
5 } mat_spr;
```

1.4.1 Caso unidimensional

Vamos considerar uma matriz unidimensional de elementos reais ($M[i], i = 1, \dots, 2^{15}$), em que todos os elementos são nulos, exceto os seguintes:

índice	2	70	816	1643	182781
valor	-1,3	-0,2	3,5	4,0	-1,0

A seguir são listados códigos exemplos de procedimentos para as operações básicas com matrizes esparsas unidimensionais. Nesses códigos, a estrutura M que representa a matriz esparsa é um vetor de tamanho máximo N do tipo `mat_spr`:

```

1 void insere( mat_spr *M,
2             int      i,
3             int      n,
4             float    v )
5 {
6     int j = 0,
7         k = 0;
8
9     while (M[j].ind < i)
10         j++;
11
12     for (k = n; k > j; k--)
13         M[k] = M[k - 1];
14
15     M[j].ind = i;
16     M[j].val = v;
17     n++;
18 }

```

```

1 void consul( mat_spr *M,
2             int      i,
3             int      n,
4             float    *v )
5 {
6     int j = 0;
7
8     if (i < 0)
9     {
10         printf( "Indice %d
11                 invalido!!!", i );
12         exit( EXIT_FAILURE );
13     }
14
15     j = lugar( M, i, n );
16     if (j == -1)
17         *v = 0.0;
18     else
19         *v = M[j]->val;
20 }

```

```

1 int lugar( mat_spr *M,
2           int      i,
3           int      n )
4 {
5     int j = 0,
6         pos = -1;
7
8     for (j = 0; j < n; j++)
9         if (M[j].ind == i)
10             pos = j;
11
12     return pos;
13 }

```

```

1 void compacta( mat_spr *M,
2              int      i,
3              int      n )
4 {
5     int j = 0;
6
7     for ( j = i; j < n-1; j++)
8         M[j] = M[j + 1];
9     M[n-1] = {-1,0.0};
10     n--;
11 }

```

```

1 void atrib( mat_spr *M,
2             int      i,
3             int      n,
4             float    v )
5 {
6     int j = 0;
7
8     if (i < 0)
9     {
10        printf( "Indice %d
11                invalido!!!", i );
12        exit( EXIT_FAILURE );
13    }
14
15    j = lugar( M, i, n );
16    if (j == -1) /* Indice i nao
17                ocorre em M */
18        if (v != 0.0)
19            insere( M, i, n, v );
20    else /* Indice i ja ocorre em M
21         */
22    {
23        if (v == 0.0)
24            compacta( M, j, n );
25        else
26            M[j].val = v;
27    }
28 }

```

Exercícios

1. Dado $M(i, j)$, ($i = 1, \dots, 10; j = 1, \dots, 10$) representado por um objeto do `vetor[0...99]` do tipo `float`, escreva um procedimento para listar os elementos da diagonal principal: $M(1, 1), M(2, 2), \dots, M(10, 10)$.
2. Uma matriz tridiagonal tem entradas a_{ij} que são zero, exceto quando $|i - j| \leq 1, 1 \leq i, j \leq n$. Escreva uma função de alocação da forma $k = f(i, j) = a_0 + a_{1i} + a_{2j}, |i - j| \leq 1$, que represente todos os elementos relevantes de uma matriz tridiagonal em $(3n - 2)$ posições consecutivas.

Solução:

- $i - 1$: número de linhas antes da linha i ;
- $3 \cdot (i - 1) - 1 = 3 \cdot i - 4$: número de elementos da antes da linha i ;
- $j - i + 1$: número de elementos antes do elemento j na linha i .

Logo, $k = b + (3 \cdot i - 4) + (j - i + 1) = (b - 3) + 2 \cdot i + j$.

3. Em uma matriz tridiagonal inversa M , de dimensão $m \times n$, todas as entradas a_{ij} são zero, exceto quando $k - 2 \leq i + j \leq k$, onde k é o máximo entre m e n . Construa uma função de alocação que represente todos os elementos relevantes da matriz tridiagonal inversa em $(3 \cdot r - 2)$ posições consecutivas, onde r é o mínimo entre n e m .
4. Suponha que você tem um matriz de ordem $m \times n$. Os únicos elementos relevantes para você são aqueles que se encontram abaixo da diagonal secundária da matriz. Mostre como representar tais elementos em um vetor de dimensão apropriada e como recuperá-los, ou seja, como usar os índices da matriz para calcular o índice do vetor.

5. Explique como é a representação linear de uma matriz n -dimensional $M(i_1, \dots, i_n)$, ($i_1 = c_1, \dots, f_1; \dots; i_n = c_n, \dots, f_n$).
6. Sugira uma estrutura de dados para o armazenamento dos elementos de uma matriz $m \times n$. A sua estrutura deve permitir que uma linha i_1 seja trocada de posição com uma linha i_2 sem que os elementos das duas linhas sejam individualmente mudados de posição. Escreva um trecho de programa em C para ler os dados da matriz e depois inverter a posição da primeira com a penúltima linha da mesma.
7. Suponha que você esteja construindo um programa em C e deseja alocar dinamicamente uma matriz de elementos do tipo inteiro e de dimensão $n \times n$. Contudo, o seu sistema computacional já está sobrecarregado e a memória disponível encontra-se muito fragmentada. Sabendo-se que existem pelo menos n blocos de memória de tamanhos suficientes para representar de n a $2 \cdot n - 1$ elementos do tipo inteiro, escreva um trecho de programa em C para alocar espaço para tal matriz. (A referência a um elemento de tal matriz deverá ser feita por alguma coisa da forma $M[i, j]$ ou $M[k]$, onde k é o endereço adequado, sem controles adicionais).
8. Um quadrado mágico é uma matriz $n \times n$ dos números inteiros de 1 até n^2 , onde a soma dos elementos em cada linha, coluna ou diagonal é a mesma. Por exemplo, se $n = 3$ a soma comum é 15:

6	1	8
7	5	3
2	9	4

Considere o seguinte algoritmo para geração de quadrados mágicos, quando n é ímpar: A posição inicial é a coluna do meio da linha superior. A partir desta posição ande sempre para uma coluna à esquerda e uma linha superior, atribuindo os números na ordem crescente (a partir de 1) nas posições vazias. Se cair em uma posição fora do quadrado, imagine o mesmo como um cilindro e continue na linha (coluna) na extremidade oposta. Caso atinja uma posição já preenchida com um número inteiro, desloque-se na mesma coluna uma linha para baixo e continue com o mesmo procedimento. Escreva um programa em C que implemente este algoritmo para gerar quadrados mágicos $n \times n$, para n ímpar.

Solução:


```

1 #include "stdio.h"
2
3 int main(int argc, char *argv[])
4 {
5     int c = 1,
6         n = atoi( argv[1] ),
7         i = -1,
8         j = ((int) (n / 2)),
9         f = n * n,
10        Q[30][30];
11
12    for (; c <= f; c++)
13    {
14        if ((c % n) == 1)
15            i++;
16        else
17        {
18            i = ((i - n) % n) + (n - 1);
19            j = ++j % n;
20        }
21        Q[i][j] = c;
22    }
23
24    for (i = 0; i < n; i++)
25    {
26        for (j = 0; j < n; j++)
27            printf( "%5d", Q[i][j] );
28        printf( "\n" );
29    }
30
31    printf( "\nSoma = %d\n\n", n * ((f +
32        1) / 2) );
33
34    return 1;
35 }

```

9. Na figura abaixo tem-se um quadrado mágico, ou seja, uma quadrado em que a soma dos elementos em cada linha, coluna ou diagonal é a mesma. Encontre valores válidos para A , B , C e D .

A	$\frac{9}{2}$	$-2,5$
B	$\frac{1}{2}$	C
$\frac{7}{2}$	D	$1,5$